CHISIMBA

Security and Abuse Prevention

A human-readable security strategy for the revived Chisimba platform

PURPOSE

This document explains how Chisimba will prevent, withstand, detect and recover from security incidents and abusive use. It describes both controls built into Chisimba and controls that must exist around it in a real deployment.

Status: Working strategy and implementation roadmap
Project: Chisimba Revival
Date: 3 August 2026
Audience: Deployers, developers, reviewers, partners and prospective adopters

Security is a maintained capability, not a product claim.

Executive summary

Chisimba will use defence in depth. Secure application code, canonical security services, hardened hosting, a carefully tuned WAF, monitoring, backups and incident response reinforce one another; none replaces the others.

The revived platform is moving away from scattered legacy security behaviour toward explicit, testable contracts. Authentication, identity, group membership and permission decisions already have canonical service boundaries. The proposed new core module, Abuse Protection (`abuseprotection`), will become the single application-level owner of throttling, transparent bot evidence and consistent allow/delay/reject decisions.

This document is deliberately honest about maturity. It describes the target security model and the work required to reach it. It is not a certification, penetration-test result or assurance that every legacy module is presently suitable for Internet exposure.

Layer	Primary purpose	Examples
Application	Prevent vulnerabilities and enforce policy	Canonical auth/permission services, CSRF, output encoding, upload rules
Abuse prevention	Control automated and repeated misuse	Rate limits, honeypots, signed form timing, progressive delay
Edge and host	Reduce attack surface and absorb hostile traffic	TLS, reverse proxy, WAF, network policy, container/OS hardening
Operations	Detect, respond and recover	Security logs, alerting, backups, restoration tests, incident playbooks
Governance and delivery	Make security repeatable and accountable	Threat models, GitHub CI/CD gates, dependency policy, review and disclosure

1. Scope and security boundary

Chisimba security includes the framework and modules, but it does not stop at the PHP process. A production service is a system comprising users, browsers, DNS, TLS termination, reverse proxy or load balancer, WAF, containers or hosts, database, file storage, outbound services, monitoring, backups and operational staff.

What Chisimba must own

- Authentication and secure session contracts; identity lookup; group membership; authorisation and contextual permission decisions.
- CSRF protection for state-changing browser requests and deliberate POST-based security actions such as logout.
- Input validation, output encoding and safe database use at every trust boundary.

- Application-level abuse decisions that understand routes, accounts, sessions and business meaning.
- Secure file-upload policy, storage naming, access checks, content handling and download response behaviour.
- Security-relevant audit events with privacy-aware content and stable event vocabulary.
- Safe defaults, secure installer behaviour and clear production configuration requirements.

What each deployment must own

- DNS, certificates, TLS policy, HTTP security headers at the correct layer and safe reverse-proxy trust configuration.
- WAF selection, rules, tuning, exclusions, alerting and a tested method to bypass or roll back bad rules during incidents.
- Operating-system, container, PHP, web-server and database patching; minimal network exposure and least privilege.
- Secret generation, storage, rotation and separation from source code and images.
- Central log retention, monitoring, alert routing and time synchronization.
- Encrypted backups, off-site or failure-domain separation, restoration testing and recovery objectives.
- Incident ownership, user communication, legal/privacy assessment and vulnerability disclosure handling.

BOUNDARY RULE

A WAF can block or slow many hostile requests and can provide valuable virtual-patching time. It cannot correctly enforce Chisimba's contextual permissions, repair insecure business logic, or prove that application code is safe.

2. Security principles

1. Services define the contract; consumers migrate to the contract. A service changes only for a proven, generally valid missing capability.
2. Every security-sensitive database table has exactly one canonical service owner.
3. Deny by default. Missing, ambiguous or failed security information never becomes permission.
4. Validate at trust boundaries and encode for the output context. Validation and output encoding solve different problems.
5. Use defence in depth, while keeping each control's responsibility explicit and testable.
6. Preserve the minimum sensitive data needed for the minimum useful period. Prefer keyed hashes where raw identifiers are unnecessary.
7. Security failures must be observable to operators without disclosing sensitive internals to users.
8. Compatibility scaffolding is temporary. It requires a removal script or checklist, named consumers and an end condition.
9. Repository source is authoritative. Disposable runtimes are rebuilt from current source and are never patched directly.
10. Claims follow evidence: tests, inspection, reproducible probes, code review and independent assessment.

3. Threat model

The threat model must be revised whenever architecture or exposure changes. The initial model assumes a public web application serving anonymous visitors, registered learners, educators, content authors and site administrators, with uploaded files and contextual course/group permissions.

Threat actor or failure	Likely objective	Representative controls
Automated bot	Spam, account creation, password guessing, scraping, resource exhaustion	AbuseProtectionService, edge rate limits, WAF, progressive delay, monitoring
Unauthenticated attacker	Exploit injection, upload or legacy endpoint weaknesses	Validation, encoding, parameterized DB access, upload isolation, patching, WAF
Authenticated low-privilege user	Access another user's or course's data	Object-level and contextual authorisation on every request
Privileged user	Misuse admin powers or exfiltrate data	Least privilege, sensitive-action checks, strong auth, audit trail, separation of duties
Compromised dependency	Execute code or expose secrets/data	Dependency inventory, pinning, provenance, updates, isolation, review
Configuration error	Expose debug output, files, database or admin surfaces	Secure defaults, deployment checks, secret scanning, environment validation
Operational failure	Lose data or remain compromised undetected	Backups, restore exercises, alerts, playbooks, immutable/off-site copies

Protected assets

- Credentials, password verifiers, session material, MFA secrets and recovery tokens.
- Personally identifiable information, learning records, assessment results and private communications.
- Course, group, role and permission integrity.
- Uploaded content and the availability and integrity of the complete `usrfiles` tree.
- Configuration secrets, signing keys, database credentials and backup material.
- Source, release artefacts, build process and audit evidence.

4. Canonical security architecture

Security decisions should flow through small, named contracts rather than being reimplemented by individual modules. Legacy façades may temporarily preserve active consumer method shapes, but the canonical service remains the source of truth.

Owner	Owns	Must not own
Authentication service	Credential verification, login outcome, authentication lifecycle	Group or permission policy
Native session service	Authenticated session state, rotation, expiry, revocation	Identity records or authorisation rules
Identity/User service	Canonical user identity and profile lookup	Authentication proof or permission grants
Group service	Groups and group membership	Permission evaluation
Permission service	Rights, grants and contextual authorisation decisions	UI visibility as a substitute for enforcement
AbuseProtectionService	Abuse evidence, counters, throttles and allow/delay/reject decisions	Passwords, sessions, permissions, email delivery or WAF policy
Application modules	Business-specific validation and authorised action	Private copies of canonical security logic

Authorisation rule

NON-NEGOTIABLE

Hiding a button, menu item or route is user-interface behaviour—not authorisation. Every protected server-side action must make an authoritative permission decision using the relevant identity, group and context.

5. The Abuse Protection core module

`abuseprotection` will be Chisimba's first new PHP 8-era core module and should establish the standard for future canonical services. It replaces the idea of a single universal CAPTCHA with route-appropriate, layered abuse controls. The basic path must work without JavaScript and without a third-party CAPTCHA provider.

Initial service contract

- Issue transparent form evidence: a signed issuance time, a nonce or form instance identifier and an optional honeypot field.
- Evaluate a named action such as `security.login`, `user.register`, `feedback.submit` or `blog.comment`.
- Combine policy with privacy-aware evidence: action, keyed IP/network hash, account hash where appropriate, session/form instance and recent outcomes.
- Return a structured decision: allow, allow-and-record, delay, reject or require a stronger application-owned step.
- Record success and failure consistently; expire counters and evidence; expose safe maintenance/cleanup operations.
- Emit security events without recording passwords, tokens, raw secrets or unnecessarily persistent raw IP addresses.

Policy examples

Action	Initial evidence	Escalation
Login	IP/network and account-keyed attempt history	Progressive delay; generic failure; operator alert for sustained attack
Registration	Signed timing, honeypot, IP/network rate	Temporary rejection; email verification in user workflow
Password recovery	IP and account-keyed rate; generic response	Delay/reject while preserving non-enumeration
Feedback	Signed timing, honeypot, IP/session rate	Delay/reject; moderation signal
Blog comment	Signed timing, honeypot, IP/account rate	Moderation queue, delay or temporary rejection

Privacy and fairness

Abuse controls can harm legitimate users behind shared networks, users with assistive technology and users on slow or unstable connections. Policies therefore need bounded windows, progressive rather than permanent penalties, accessible messaging, operator visibility and a way to recover from false positives. Minimum-completion timing should be conservative and should contribute evidence rather than operate as a lone absolute test.

Legacy CAPTCHA retirement

11. Inventory every active CAPTCHA consumer and its risk; defer obsolete modules explicitly rather than silently.
12. Integrate `abuseprotection` with one low-risk anonymous form and test bypass, replay, accessibility and false positives.
13. Migrate Blog, feedback, registration, password recovery and authentication in separate bounded changes.
14. Confirm that no active consumer calls the legacy endpoint or expects `captchaDiv` replacement.
15. Remove the image generator, refresh endpoint, tables/configuration and JavaScript loaders using an explicit removal checklist.

6. Application security controls

Authentication and account lifecycle

- Use modern password hashing through supported PHP facilities; make cost configurable and rehash on successful authentication when policy changes.
- Apply generic responses where a specific answer would reveal whether an account exists.
- Rotate session identifiers after authentication and privilege changes; expire and revoke sessions deliberately.
- Protect password reset and other recovery tokens as short-lived, single-use credentials.
- Require stronger authentication for privileged users when a tested MFA capability is introduced; do not bolt it onto the first-install boundary.

Browser sessions and CSRF

- Use Secure, HttpOnly and appropriate SameSite cookie attributes; do not place session identifiers in URLs.
- Require CSRF tokens on state-changing browser actions. SameSite is a supporting control, not the only defence.
- Do not change state through GET requests. Logout and destructive/admin actions use deliberate POST requests with CSRF validation.
- Set idle and absolute expiry appropriate to the deployment and revoke server-side state on logout where supported.

Input, output and database access

- Accept only expected parameters and types. Treat headers, cookies, uploaded filenames, rich text and stored content as untrusted.
- Use parameterized database operations. Existing `dbTable` responsibilities must be understood and verified rather than bypassed with scattered SQL.
- Encode output for its actual context: HTML text, attribute, URL, JavaScript, CSS, JSON or email. Rich HTML requires a maintained allow-list sanitizer.
- Return safe error messages to users; keep diagnostic details in protected logs.

File uploads and content delivery

- Allow only business-required types and sizes; validate extension, detected type and file structure where feasible.
- Generate server-side storage names. Never allow an uploaded name to choose an executable or traversable path.
- Store uploads outside executable web roots where possible and serve them through authorisation-aware delivery.
- Apply malware/content scanning where the deployment risk warrants it; quarantine until a decision is available.
- Set safe download headers and isolate active content. Image and document processing libraries are part of the attack surface.

7. Platform, edge and deployment security

WAF and reverse proxy

A production deployment should normally place Chisimba behind a maintained reverse proxy and, where exposure and resources justify it, a WAF. An open implementation can use ModSecurity-compatible engines with the OWASP Core Rule Set; a managed WAF may be preferable where operational capacity is limited.

- Start in detection or low-blocking mode, observe legitimate Chisimba traffic, then tune and promote rules deliberately.
- Version-control local exclusions and document the endpoint, reason, owner and expiry/review date for each exclusion.
- Rate-limit high-risk routes at the edge as a coarse first layer; retain application-level policy because the WAF lacks account and course context.
- Trust forwarded client IP and scheme headers only from explicitly trusted proxies. Prevent clients from spoofing the identity used for rate limits or audit.

- Alert on rule spikes and repeated targets. Test emergency rule rollback and origin protection so attackers cannot simply bypass the WAF.

Host, container and network

- Run supported PHP, database, web-server and base-image versions; remove unused packages and services.
- Run processes as non-root with read-only application code where practical; grant write access only to named runtime paths.
- Expose only required ports. The database and management surfaces must not be publicly reachable.
- Separate development diagnostics from production; disable display of PHP errors and development tooling in production.
- Use resource limits and health checks, but avoid policies that convert routine load into data loss.

Secrets and configuration

- Do not commit secrets, ship default credentials or bake production credentials into images.
- Use high-entropy environment-specific secrets and document rotation without requiring data loss.
- Validate production configuration at startup or deployment: HTTPS, cookie policy, debug state, writable paths and key material.
- Restrict and audit administrative access to deployment, database, backup and DNS systems.

8. Logging, detection and response

Logs should let an operator answer who attempted what, against which target or context, when, from which privacy-safe source classification, and with what outcome. They should not become a second sensitive database.

Event family	Examples	Alert considerations
Authentication	Success, failure, lock/delay, reset requested/completed, session revoked	Bursts, distributed guessing, privileged account activity
Authorisation	Denied protected action, grant/revoke, role/group change	Repeated denials, privilege changes, impossible scope
Abuse	Rate threshold, honeypot, replay, timing anomaly, rejection	Action-specific spikes, changing sources, false-positive rate
Content/files	Upload rejected/quarantined, sensitive download denied	Executable/active content, mass access, scanner failure
System	Configuration failure, dependency error, backup/restore result	Logging stopped, repeated fatal errors, failed recovery control

Incident response minimum

16. Triage and preserve evidence; establish an incident owner and time line.
17. Contain without destroying evidence or silently losing user data and files.

18. Revoke exposed sessions, tokens or credentials and isolate affected components as appropriate.
19. Determine scope, entry point, affected data and whether persistence remains.
20. Recover from a known-good state, validate integrity and increase monitoring.
21. Communicate to affected parties and authorities where obligations apply.
22. Complete a blameless post-incident review and track corrective actions to closure.

9. Secure development and assurance

- Threat-model new services and high-risk changes before implementation; record trust boundaries and abuse cases.
- Use small, evidence-driven migrations with narrow diffs, expected counts, linting, rollback and runtime tests.
- Add unit and contract tests for canonical services and integration tests for critical flows: install, login, authorisation, logout, account recovery, uploads and restoration.
- Scan dependencies and source for known vulnerabilities and secrets; maintain an inventory of shipped third-party code.
- Require review for security-sensitive changes and avoid authors approving their own highest-risk work where project capacity permits.
- Use OWASP ASVS 5.0 as the application verification requirements catalogue. Select and document an appropriate target level rather than claiming blanket compliance.
- Commission independent penetration testing before inviting material public or institutional use, and after major security-architecture changes.
- Publish a supported-version policy and a responsible vulnerability disclosure channel before broad deployment.

GitHub CI/CD security pipeline

GitHub should be the controlled path from reviewed source to a reproducible release and, where deployment is automated, to the target environment. The pipeline is not a substitute for review or operational judgment; it makes agreed controls consistent, visible and difficult to bypass accidentally.

- On every pull request, run PHP syntax and compatibility checks, unit and contract tests, critical-flow smoke tests, static analysis, dependency vulnerability checks, secret detection and checks for prohibited generated or runtime-only changes.
- Protect release branches with required status checks and review. Security-sensitive changes should require an appropriate reviewer, and direct pushes or self-approved high-risk changes should be prevented where project capacity permits.
- Build release artefacts once from a pinned source revision; record the commit, workflow version, dependency lock state, test results, checksums and software bill of materials. Promote the same verified artefact between environments instead of rebuilding it differently for production.
- Use short-lived, least-privilege deployment credentials and protected GitHub environments. Keep production secrets out of repositories, workflow logs, build artefacts and untrusted pull-request contexts; require deliberate approval for production deployment.
- Run post-deployment health and security smoke tests, retain deployment evidence, and support a tested rollback to a previously verified artefact without overwriting databases or user files.
- Adopt gates progressively: establish the legacy baseline in report-only mode, then block new regressions immediately and make release-critical failures blocking before public deployment. Existing debt must remain visible, owned and time-bounded rather than normalized away.

Security evidence pack

When Chisimba is presented to a partner, the strongest answer is evidence rather than adjectives. A release-ready evidence pack should contain:

- Architecture and data-flow diagrams with trust boundaries.
- Current threat model and risk register with accepted, mitigated and deferred risks.
- ASVS mapping with requirement status and test evidence.
- Dependency inventory/SBOM and vulnerability-handling policy.
- GitHub CI/CD run evidence: required checks, reviews, source revision, artefact checksum, SBOM, approvals, deployment result and rollback record.
- Automated test results and reproducible smoke-test procedures.
- Independent assessment summary and remediation status.
- Deployment hardening guide, backup/restore evidence and incident-response contacts.

10. Prioritised roadmap

Priority	Outcome	Exit evidence
P0 — before public exposure	Close known PHP/runtime fatals; canonical auth/session/permission enforcement; CSRF on state change; safe errors; secure production config; backups and tested restore; patch supported stack	Clean critical-flow tests, configuration check, restore record, no known critical/high exploitable issue
P1 — first new service	Create `abuseprotection`; prove on one anonymous form; establish security event vocabulary and cleanup	Contract tests, bypass/replay tests, privacy review, operational counters
P1 — active consumers	Migrate Blog, feedback, registration, recovery and login; retire active legacy CAPTCHA consumers	Per-consumer tests; explicit removal checklist; no active endpoint references
P1 — deployment baseline	Reverse proxy/WAF guidance, trusted proxy policy, security headers, least privilege, central logging and alerting	Hardened reference deployment; tuned rules; alert and rollback drill
P1 — secure delivery pipeline	Establish GitHub pull-request checks, protected releases, reproducible artefacts, SBOM/provenance, protected deployment environments and rollback	Required checks and review policy; retained pipeline evidence; verified artefact promoted to a test environment; rollback drill
P2 — systematic assurance	ASVS 5.0 mapping, dependency/SBOM process, upload review, legacy-module risk classification	Tracked requirement matrix and remediation queue

Priority	Outcome	Exit evidence
P2 — external confidence	Independent penetration test and documented remediation; disclosure and supported-version policies	Assessment report/summary and closed findings
P3 — maturity	MFA for privileged accounts, advanced anomaly signals, repeated exercises and measured security objectives	Operational metrics and periodic review evidence

11. Questions Chisimba should be ready to answer

- How are authentication, identity, groups and permissions separated, and where is each enforced?
- Can a user access another user's or course's objects by changing an identifier? What tests prove the answer?
- How are anonymous forms protected without inaccessible third-party CAPTCHA dependence?
- Which actions are rate-limited at the edge and which are evaluated inside Chisimba?
- How are uploaded files validated, stored, authorised, scanned and served?
- How quickly can a vulnerable dependency or WAF rule be updated, tested and deployed?
- Which GitHub checks and approvals are mandatory, what artefact was deployed, and can its source revision, dependencies, SBOM and test evidence be reproduced?
- What security events are collected, who receives alerts and how long are records retained?
- When was the last successful restoration of the database and complete file tree?
- Which modules are production-supported, paused, obsolete or still awaiting security review?
- What independent assessment has been performed, what was found and what remains open?

12. Current Chisimba Revival position

STATUS AS AT 3 AUGUST 2026

The canonical authentication, identity, group and permission direction is established and key install/login/authorisation/logout flows have been proven during revival work. Significant legacy consumer migration and module review remain. The new Abuse Protection module is proposed but not yet implemented. Production security claims must therefore be tied to the exact release, enabled modules, deployment configuration and available test evidence.

Recent engineering work has demonstrated the project's intended security discipline: circular initialization was traced and fixed at its architectural cause; PHP 8 declaration and syntax failures were repaired with hash-pinned, rollback-safe changes; runtime rebuilding preserves the complete file tree; and HTTP success is not accepted when response bodies or logs contain hidden fatal errors. These practices should become release policy rather than remaining ad hoc migration techniques.

13. Reference framework

The following public standards and guidance anchor this strategy. They are references for requirements and verification, not substitutes for Chisimba-specific threat modelling.

- **OWASP Application Security Verification Standard (ASVS) 5.0:** https://owasp.org/www-project-application-security-verification-standard/
- **OWASP Top 10: 2025:** https://owasp.org/www-project-top-ten/
- **OWASP Cheat Sheet Series:** https://cheatsheetseries.owasp.org/
- **OWASP Session Management Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- **OWASP CSRF Prevention Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- **OWASP File Upload Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- **OWASP Core Rule Set:** https://owasp.org/www-project-modsecurity-core-rule-set/
- **NIST Cybersecurity Framework 2.0:** https://www.nist.gov/cyberframework

Document maintenance

Review this document at each security milestone, before a production release, after a material architecture change and after any significant incident. Replace proposed statements with verified-state statements only when evidence is retained. Record major revisions in source control alongside the Chisimba Revival documentation.